

CSE 508
NETWORK SECURITY — PRACTICE MIDTERM 2 (EASY) —
SOLUTIONS

SPRING 2026 — OPEN BOOK

100 Points

Instructor’s model solutions. Where more than one answer is acceptable, the main alternatives are listed.

Section 1 — Mandatory Questions

1.1 Prefix Hijacking and a Simple Defense

(a) The attacker announces, from AS 64500, that *it* originates 198.51.100.0/24. Its AS_PATH therefore looks like a short one-hop path containing just AS 64500 (plus whatever ASes the announcement is propagated through on its way to any given listener). It is a “classic” origin hijack: AS 64500 pretends to own a prefix it does not own. (A more aggressive version announces a longer/more-specific prefix such as 198.51.100.0/25; accepted as a correct alternative for this easy question.)

(b) BGP has no built-in authentication of who may originate a prefix, so AS 64700 cannot tell the fake announcement from the real one just by looking at it. Once AS 64700 has both routes, normal BGP route selection kicks in: routers prefer the *shortest* AS_PATH, and the path through AS 64500 may be as short as or shorter than the real path through AS 64600. If the path length is a tie, routers prefer the route learned from the neighbor they are configured to prefer. Either way, the fake route can easily look better than the real one. (Other acceptable reasons: BGP prefers more-specific prefixes over less-specific ones, so a /25 hijack always wins over the real /24.)

(c) Any *one* of the following is enough:

- **Publish a ROA and encourage upstreams to do Route Origin Validation (ROV):** the RPKI ROA signed by Acme’s registry says “only AS 64600 is allowed to originate 198.51.100.0/24.” Upstreams that perform ROV will then mark the attacker’s route as “Invalid” and drop it.
- **Prefix filtering by upstreams:** AS 64700 can refuse to accept announcements from AS 64500 for prefixes that AS 64500 is not listed as owning in public data (e.g. in a prefix list maintained by the operator).

(Note: neighbor authentication (MD5) and TTL-check are BGP-session defenses against an off-path attacker hijacking a particular session; they do **not** stop a legitimate peer from announcing somebody else’s prefix, so they are not acceptable answers here.)

1.2 Why a Server Certificate Prevents Impersonation

(a) The certificate is a signed statement by a CA that a specific public key belongs to `bank.example`. Alice’s browser already trusts that CA’s root key, so it can verify the signature. During the TLS handshake the bank proves, using its matching *private* key, that it is the party named in the certificate. Mallory does not have the bank’s private key, so she cannot produce the proof — all she can produce is a certificate for some *other* name (or an unsigned key), which Alice’s browser rejects.

(b) If Alice’s browser accepts any public key without checking a certificate, Mallory performs a standard on-path man-in-the-middle: she terminates Alice’s TLS handshake on one side using her own key pair, and opens a separate TLS session to the real bank on the other side. Alice’s browser trusts Mallory’s key; the bank trusts Mallory’s connection as just another client. Mallory decrypts everything in the middle and optionally modifies it.

(c) Any *one* of the following PKI-level failure modes:

- A CA is compromised (or coerced) and issues a fraudulent certificate for `bank.example` to an attacker.
- A benign CA is *tricked* into issuing a certificate for `bank.example` (for example, via a short-lived BGP hijack that redirects the CA’s domain-validation probes).

Any answer that identifies a failure of the Web PKI’s trust root (rather than a correct PKI producing a correctly-validated cert) is acceptable.

Section 2 — Long Questions

2.1 Outbound SSH Defeats a Web Filter

(a) Bob opens an SSH connection from his laptop to `home.example`. On top of that SSH connection he establishes a *local* port-forwarding tunnel: he tells his SSH client “anything that connects to port *X* on my laptop should be delivered, through this SSH session, to the blocked news site’s address on port 80 (or 443), with `home.example` making that outbound connection on his behalf.” He then points his browser at `localhost:X`; the bytes travel encrypted inside SSH to `home.example`, and `home.example` — which is not behind the university firewall — fetches the news site. (An equally correct answer uses a *dynamic* SSH tunnel that exposes a local SOCKS proxy, so the whole browser is routed through `home.example`.)

(b) The firewall sees one long-lived outbound TCP connection from Bob’s laptop to `home.example` on port 22. That is exactly the SSH traffic the policy allows. The payload is encrypted by SSH, so the firewall’s header-only view never shows the blocked site’s hostname or URL. Deep packet inspection cannot peer inside an SSH stream either, because it is end-to-end encrypted.

(c) Any *one* of the following, with a plausible trade-off:

- **Restrict outbound SSH to a short list of allowed destinations** (e.g. GitHub, specific lab servers). Downside: breaks legitimate uses (students SSHing to their own

VMs), creates a maintenance burden as the allow-list has to be curated, and provides no defense against a user who happens to have shell access on any of the allow-listed hosts.

- **Move to a default-deny egress policy + a filtering proxy** so that the only allowed outbound protocols are those the proxy can parse. Downside: operationally heavy, breaks many applications, and privacy trade-off for users.
- **Host-based / endpoint controls** (only university-managed devices allowed on the network, outbound tunneling apps blocked on the endpoint). Downside: cannot cover personal devices and is a much larger administrative investment.

2.2 ROA and ROV

(a) (i) A ROA is a signed record that binds an IP prefix to the AS number that is authorized to originate it. (ii) A router doing ROV compares each incoming BGP announcement's origin AS and prefix against published ROAs. The announcement is labeled *Valid* if a ROA authorizes that AS for that prefix, *Invalid* if a ROA exists for the prefix but names a different origin AS (or disallows the specificity), and *NotFound/Unknown* if no ROA covers the prefix. A typical policy drops Invalid routes.

(b) A simple origin hijack where an attacker AS claims to originate a prefix that actually belongs to someone else: ROV sees the origin AS does not match the ROA and labels the announcement Invalid, so any ROV-enforcing router drops the route. The attacker's announcement then fails to propagate through those routers and cannot attract traffic.

(c) Any *one*:

- ROA/ROV validates only *origin* AS and prefix, not the rest of the `AS_PATH`; an attacker can still forge the rest of the path to claim a direct connection to the real origin AS.
- ROAs are not universally published and ROV is not universally enforced, so many routers still accept Invalid routes.

2.3 ACME HTTP-01 and Multi-Perspective Validation

(a) To serve the expected content at `http://blog.example/.well-known/acme-challenge/<token>`, you have to be the party whose web server answers when the ACME server resolves `blog.example` and opens a TCP connection to it on port 80. Being that party is essentially the operational definition of “controlling the domain”: you control either the name's DNS record or the IP it points to, plus the web server running there. Additionally, the response is bound to the ACME account's public key (via the key authorization), so an unrelated party who merely sniffs the token cannot reuse it — they would also need the account's private key.

(b) If the attacker briefly hijacks the BGP prefix containing `blog.example`'s IP, then for the duration of the hijack a router near the ACME server may route traffic for that IP to the attacker's AS. The ACME server's HTTP GET for the challenge URL therefore lands on the attacker's web server, which can serve whatever content it wants. The attacker has registered its own ACME account, so it can serve its own correct key authorization

and pass the HTTP-01 challenge — and the CA will issue a real, CA-signed certificate for `blog.example` to the attacker.

(c) With multi-perspective validation, Let's Encrypt fetches the challenge URL from several geographically diverse vantage points. A short-term hijack that successfully poisons the route as seen from *one* path typically does not succeed from the others, so some validators still reach the legitimate server and the observations disagree; the ACME server treats this as a failure. The attacker would have to hijack the prefix from the perspective of *every* vantage point simultaneously, which is dramatically harder.

2.4 DNS Cache Poisoning at a High Level

(a) Eve races the legitimate authoritative server to be the *first* to deliver a response that the resolver accepts. To win, she sends many forged UDP packets that look like responses to the resolver's outstanding query. For the resolver to accept one of them, Eve must get several fields right: the destination UDP port on the resolver (the resolver's source port for the query), the 16-bit transaction ID (TXID), and the query section (name, type, class) copied from the original query. Because these fields (especially TXID) are small, Eve can attempt many forgeries in parallel before the real answer arrives; if one of them matches, the resolver caches her bogus mapping.

(b) If the resolver uses a *random* UDP source port for each outgoing query (instead of a fixed one), Eve must guess both the 16-bit TXID *and* the source port. Per the lecture, this raises the entropy she has to brute-force from 16 bits to about 27 bits (16 TXID + 11 source-port), making a successful race dramatically less likely in the short window before the real reply arrives.

(c) 0x20 randomization uses the fact that DNS name matching is case-insensitive but most resolvers copy the exact case of the queried name into the response. The resolver randomizes the case of letters in its outgoing query (e.g. `WwW.exAmple.coM`) and requires the response to match byte-for-byte. An off-path attacker who cannot observe the query cannot guess the case pattern. One reason it does not always help: not every authoritative server or resolver preserves the case in the reply (some lower-case everything), and a name with no letters (or with only digits) has no entropy to add.

2.5 Perimeter Firewall vs. Zero Trust

(a) For example: an employee clicks a phishing link on their laptop while inside the corporate LAN (or while connected through the VPN). The malware now runs on a device that the perimeter model considers “inside,” and therefore has a direct network path to every internal service — file shares, databases, admin consoles — because those services trust “inside” traffic without further authentication. One compromised laptop therefore yields lateral movement to everything. Another accepted answer: a stolen/forgotten VPN credential gives the attacker the same implicit trust.

(b) Zero Trust treats the network itself as untrusted. Being on the corporate LAN or VPN grants you *nothing* by default; every access to a resource must be authenticated (who is the user? what is the device? is its posture acceptable?) and authorized against an explicit policy for that resource.

(c) Any *one*: the rollout cost and complexity is high (every resource must be placed behind a policy-enforcement point and wired into an identity system); user friction (more authentication prompts, device-posture checks); the policy engine becomes a new high-value target and single point of failure; and legacy applications that assume “on the LAN” trust may need to be retrofitted.

Section 3 — Short Questions

3.1 Stateless vs. Stateful Packet Filter

Example: “Allow outbound connections from inside and all their return traffic, but block any connection initiated from outside.” A stateful firewall remembers that an outbound TCP connection was established and allows matching inbound packets in; a purely stateless filter cannot do this cleanly because it cannot tell a reply packet from an unsolicited one without flag-tricks that attackers can spoof.

3.2 “NAT is Not a Firewall”

NAT was introduced to conserve IPv4 addresses, not to enforce a security policy. It blocks unsolicited inbound connections only because it has no mapping for them — a side effect of address translation. It does not validate TCP state transitions, does not block outbound traffic, and any inbound hole (e.g. port forwarding, UPnP, a currently active outbound flow) is a path in. A proper firewall applies an *explicit* policy rather than relying on this accident.

3.3 Why the iBGP Split-Horizon Rule Exists

iBGP does *not* prepend the AS number to the `AS_PATH`, so the usual BGP loop-detection trick (“drop any route whose `AS_PATH` contains my own AS”) does not work for routes bouncing around inside one AS. Forbidding iBGP routers from re-advertising iBGP-learned routes prevents those internal loops.

3.4 MD5 on BGP Sessions

It protects the *session* between two configured BGP neighbors: an off-path attacker cannot inject spoofed TCP segments (resets or forged UPDATE messages) into that session without the shared secret. It does *not* protect against a legitimate (but misconfigured or malicious) peer announcing bogus prefixes — the peer is authenticated, but nothing about the content of its announcements is.

3.5 Symmetric vs. Asymmetric Keys: the Distribution Problem

Symmetric encryption requires both parties to already share the same secret key. If the two parties have never met and are communicating over an insecure channel, there is no way to agree on that key without leaking it. Public-key cryptography lets each party publish a

public key and keep a matching private key secret, so strangers can encrypt to each other (or verify each other's signatures) without a prior shared secret.

3.6 What a Valid SCT on a Certificate Means

A valid SCT establishes only that the certificate was *submitted* to a Certificate Transparency log and that the log operator promised to include it within its maximum merge delay. It does *not* establish that the certificate is trustworthy, that the domain owner authorized it, or even that the log actually did include it yet — a malicious CA can still submit a fraudulent certificate and obtain an SCT.

3.7 SYN Flooding and SYN Cookies

- (i) It exhausts the server's half-open connection (SYN-queue) memory: each SYN causes the server to allocate per-connection state while waiting for the final ACK that never comes.
- (ii) SYN cookies avoid allocating any state for a new SYN. The server encodes the necessary connection information into the initial sequence number it returns in the SYN/ACK — if and when the client's ACK comes back, the cookie is decoded and validated, and only then is state created.

3.8 UPnP on Home Routers

A malicious program (or compromised device) on the home LAN can use UPnP to ask the router to open inbound ports mapping directly to it — no password is required. That lets the malware expose services on the open Internet (for example, a backdoor listener on the infected machine or on other LAN devices), bypassing the implicit deny-inbound of the NAT.